claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a9da9280b89fab863.jsonl`
- SHA-256: `b527dfff54271795233bee1c340c0c961e4a9ce160c19321080c0bae09dd3a5b`
- Source modified: `2026-07-02T02:55:31+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T02:47:11.984Z)

CONTEXT (read carefully):
- You are one auditor in a multi-agent tech-debt/critical-fix audit of the khelsutra-guru multi-repo workspace at C:/Users/avidu/Projects/khelsutra-guru. Today is 2026-07-02. All git repos were just pulled to current origin master/main.
- Repos: badminton-highlight-indexer (the ENGINE, Python, ~1500 tests, org Khelsutra), khelsutra (SPA web/ + marketing site/, TS/React, org avidullu), sports-data-collector, rally-annotator, sports-obsreport (small Python repos). Plus two NON-git dirs: "Annotation Setup", khelsutra-screencast.
- HARD RULES: READ-ONLY. Do NOT modify/create/delete any file, do NOT create branches/worktrees, do NOT pip install, do NOT run the full test suite (CI is trusted on master). You MAY run read-only git and gh commands (gh is authenticated), ruff/mypy IF already installed, and fast read-only scripts.
- EVERY claim must cite file:line (or a gh URL). A claim without file:line evidence is a hypothesis - mark it as such or drop it. The codebase drifts between sessions: re-verify anything you believe from docs against actual code.
- OWNER-GATED areas (flag as owner_gated=true, still report): alpha/serving go-live + Cloudflare dashboard work, Studio P10/P11 (corpus promotion + train/reeval), product/strategy/licensing decisions, real GPU/cloud spend, counsel ToS/DPA, repo rename off "badminton".
- Known-open items from project memory (VERIFY current state, don't trust blindly): engine PR #390 (D1 split main.py, open since 06-26) · audit doc docs/CODE_CLEANUP_AUDIT_2026-06-24.md remaining items D1, D13 (ByteTrack->trackers), Phase-3 (B3/B4/B6/B7/B8/D5/D6/D8-D10/O3/O4) · docs/DECISION_SNAPSHOT_REGRESSION.md next=P1 · docs/GOLDEN_REGRESSION_FIXTURES.md backlog M4/P0-rename/O1/O2 · engine issues #340 (Gemini re-bill), #332 (NSFW preflight), #349 (CPU-bound decode), #384 (CI single-source) · khelsutra issues #97-#114 UX/alpha backlog · DO droplet RETIRED 2026-06-25 (docs may still reference it; api.khelsutra.guru CNAME/CF-Access cleanup was pending).
- Categorize each finding: critical-fix (latent bug/correctness/data-loss/cost-leak), security, tech-debt (structure/duplication/dead code), test-gap, ci-infra, doc-rot, hygiene.
- Severity: P0 = latent bug or cost/data/security risk reachable in normal use; P1 = high-value debt blocking velocity or a stale-open thread; P2 = worthwhile cleanup; P3 = nice-to-have (report max 3 of these).
- Be selective: max ~12 findings, each genuinely worth acting on. Note tracked_in when an issue/PR/doc already tracks it (tracked is NOT done - still report). Your final message is data for an orchestrator, not prose for a human.

YOUR DIMENSION: performance + resource-lifecycle debt in the ENGINE. Repo: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer.
Verify at file:line the current state of: issue #349 (WASB streaming decode CPU-bound/single-threaded - where, what would parallel decode take); issue #331 (hosted processing CPU-bound - per-phase breakdown doc'd?); issue #330 (MD5 segmentation reuse - is any dedup implemented?); temp/cache lifecycle (the cache-hygiene policy: keep_frames default, tools/cleanup_caches.py, upload retention sweep #301 - all still wired?); ffmpeg subprocess handling (timeouts? zombie risk?); SQLite contention posture (busy_timeout/WAL - sports-data-collector just fixed theirs in

#53; does the ENGINE's DB layer have the same gap? backend/**/db*.py or storage). Report gaps with concrete failure scenarios.

2. user (2026-07-02T02:47:22.058Z)

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
master...origin/master

3. user (2026-07-02T02:47:27.470Z)

{"nComments":0,"state":"OPEN","title":"WASB native: decode is CPU-bound \u0026 single-threaded in streaming mode ? L4 GPU ~80% idle (no NVDEC/parallel decode)","updatedAt":"2026-06-23T16:24:58Z"}
{"state":"OPEN","title":"[perf] Hosted processing is CPU-bound (ffmpeg/cv2) ? measured per-phase breakdown + levers (GPU/NVENC, parallel extraction, async compile)","updatedAt":"2026-06-22T14:14:15Z"}
{"state":"OPEN","title":"[optimization] MD5 segmentation reuse ? skip Gemini re-segmentation when the same video (video_id) is re-uploaded","updatedAt":"2026-06-22T14:11:32Z"}
{"state":"OPEN","title":"T11 follow-up: run the upload retention sweep as an automated scheduled job (hosted alpha)","updatedAt":"2026-06-21T17:34:32Z"}

4. assistant (2026-07-02T02:47:32.877Z)

All four issues are still OPEN. Now let me map the repo structure and locate the relevant code.

5. user (2026-07-02T02:47:35.519Z)

CLAUDE.md
README.md
__pycache__
artifacts
backend
backend.db
config.json
deploy
docs
eval_baselines
indexer.db
mypy.ini
output
pyproject.toml
pytest.ini
rally_index.db
requirements.txt
ruff.toml
run_all_tests.py
run_eval.py
run_regression.py
scratch
scripts
tests
tools
training
__pycache__
annotations
api
assets
audio
billing.py
compute
config
detectors

```
eval
features
flywheel.py
infrastructure
main.py
pipeline
providers
reporting
results.py
sports
stitcher
storage
tools
ui
utils
worker
__init__.py
__pycache__
check_md_links.py
cleanup_caches.py
generate_highlights.py
leak_scan.py
```

6. user (2026-07-02T02:47:36.558Z)

```
backend\billing.py:22:WALL_SECONDS = "wall_seconds"  # total wall time (separate from GPU so
idle/sync isn't billed as GPU)
backend\billing.py:29:_RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
docs\CODE_MAP.md:7:[Omitted long matching line]
docs\CODE_CLEANUP_AUDIT_2026-06-24.md:90:| **O3** | MEDIUM |
`backend/infrastructure/database.py` | SQLite WAL + busy_timeout but concurrent writers (CLI +
server) risk `SQLITE_BUSY`. | 3 |
docs\HOSTING_PLAN.md:37:                       ? async jobs (#16), SQLite WAL, output/ on disk
tests\test_async_jobs.py:451:# --- real executor (cross-thread SQLite/WAL sanity)
---------------------------------
docs\PACKAGING_PLAN.md:115:| P2d | **DB upgrade contract** ? `SCHEMA_VERSION` + downgrade guard
(`SchemaTooNewError`) + backup-before-migrate (`<db>.bak-v<old>`, WAL-complete) +
`backup_database()` export. Byte-identical for fresh/current DBs. *(uninstall manifest deferred
to P2b-install.)* | ? | No | ? | [#307](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/307) |
docs\UI_ALPHA_EXECUTION_PLAN.md:44:- [ ] `python run_all_tests.py` green; UI polling vs SQLite
WAL sanity; WASB/CLI paths untouched
scripts\nightly_reelcount.py:90:    usage = {billing.WALL_SECONDS: float(billed_seconds)}
scripts\nightly_reelcount.py:91:    rates = {billing.WALL_SECONDS: rate_per_sec}
backend\infrastructure\database.py:36:        handles on the WAL sidecars under Windows).
Internal methods use this.
backend\infrastructure\database.py:46:        # busy_timeout: wait (not fail) on a locked DB ?
needed once the async job model
backend\infrastructure\database.py:52:        # WAL allows concurrent readers during a write;
busy_timeout bounds lock waits.
backend\infrastructure\database.py:53:        conn.execute("PRAGMA busy_timeout = 30000")
backend\infrastructure\database.py:55:            conn.execute("PRAGMA journal_mode = WAL")
backend\infrastructure\database.py:57:            pass  # e.g. a read-only/networked FS that
can't do WAL ? degrade gracefully
backend\infrastructure\database.py:157:        Checkpoints the WAL into the main file first so
the copy is complete. Best-effort: a backup
backend\infrastructure\database.py:180:        Checkpoints the WAL first so ``dest`` is self-
contained. Returns ``dest``."""
backend\infrastructure\database.py:979:                usage.get(billing.WALL_SECONDS),
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:77:- 2026-06-10 hardening
(detector keying by video_id/MD5, re-ingest destroy-AFTER-confirm via watermarks+prune,
WAL+busy_timeout, typed wasb config, ExecutionPolicy, path safety helpers, single
`compute_video_id`, trajectory twin dedup into `TrajectoryHybridSegmenter`, person cue
extraction to reusable producer) is visibly landed and tested.
tests\test_import_smoke.py:61:    """THE IMPORT WALL (ADR-008): `import backend.main` must load
```

```
no training.* module.
tests\test_import_smoke.py:76:        "print('WALL_OK')\n"
tests\test_import_smoke.py:84:    assert result.returncode == 0 and "WALL_OK" in result.stdout,
(
tests\test_reliability_hardening.py:1:"""Reliability hardening: WSL/provider timeouts, SQLite
WAL+busy_timeout, chunk-step guard."""
tests\test_reliability_hardening.py:30:def test_sqlite_wal_and_busy_timeout(tmp_path):
tests\test_reliability_hardening.py:34:        assert conn.execute("PRAGMA
journal_mode").fetchone()[0].lower() == "wal"
tests\test_reliability_hardening.py:35:        assert conn.execute("PRAGMA
busy_timeout").fetchone()[0] == 30000
```

7. user (2026-07-02T02:47:44.741Z)

```
backend\infrastructure\database.py:34:        ``with sqlite3.connect(...)`` alone is only a
commit/rollback scope ? it
backend\infrastructure\database.py:48:        conn = sqlite3.connect(self.db_path, timeout=30.0)
```

8. user (2026-07-02T02:47:45.910Z)

```
No files found
```

9. user (2026-07-02T02:47:52.346Z)

```
Found 30 files limit: 30
backend\pipeline\detectors\base.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\wasb_runner.py
backend\eval\served_gate_a.py
backend\main.py
backend\config\models.py
backend\eval\rally_seg_eval.py
backend\eval\windowing.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\worker\__main__.py
backend\utils\run_telemetry.py
backend\reporting\harvest.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\detectors\court_gate.py
backend\pipeline\detectors\device.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\native_wasb_runner.py
backend\pipeline\detectors\person_detector.py
backend\pipeline\detectors\stub_runner.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\__init__.py
backend\pipeline\audio\hit_detector.py
backend\eval\rally_seq_proto.py
backend\eval\serve_contrast.py
backend\eval\training.py
backend\eval\audio\e1_probe.py
backend\eval\calibrate_local.py
backend\eval\calibrate_wasb.py
```

10. user (2026-07-02T02:47:53.623Z)

Summary

WASB native detection on the L4 is **decode-bound on the CPU**, not GPU-bound. On the streaming path the frame decode runs **single-threaded** (`num_workers` is forced to 0), so the L4 sits mostly idle (~20% average, bursting 0?100?0) while one CPU core does HEVC/H.264 decode via `cv2.VideoCapture`. There is **no GPU/NVDEC decode path** anywhere in the engine. Throughput on

a 1080p HEVC clip was ~2× realtime (5:52 video ? ~12.5 min wall-clock).

Evidence (live, on the L4 VM)

- `nvidia-smi dmon` during a run: `sm%` bursts `0 ? 85 ? 100 ? 25 ? 0` ? the L4 does a forward pass, then waits on the next decoded batch. Average ? 20%.
- The `wasb_infer.py` process ran at ~198% CPU; GPU temp 47?77 °C, ~4.2 GB resident.
- 5:52 (352 s) 1080p HEVC, 691 MB ? started 13:50, finished 14:02 (~12.5 min).
- Installed OpenCV in the wasb env (`4.13.0`, headless PyPI wheel) reports `hasattr(cv2, "cudacodec") == False` ? no GPU decode even if code asked for it. (System `ffmpeg` *does* list `cuda` as a hwaccel, but the engine never routes decode through it.)

Root cause

1. **All decode is CPU `cv2.VideoCapture`** (OpenCV's bundled ffmpeg). No `cv2.cudacodec` / NVDEC / `-hwaccel cuda` / decord-GPU anywhere in `backend/`.
2. **Streaming forces single-process decode.** `wasb_infer.run()` sets `loader_workers = 0 if streaming else num_workers` (the in-memory frame store + the `cv2.imread` shim can't be pickled to DataLoader workers). So the fast (no-PNG) path is also the *least parallel* one.
3. The disk-frames path (`stream_video=false`) **can** use `num_workers>1` to parallelise decode/read and keep the GPU fed, but pays a full PNG extraction first (slow + disk). The 2026-06-20 deploy report already noted streaming "starves the GPU" on long clips and recommended the disk path for them.

Options (in rough order of effort)

- **Config-only mitigation (testable now, a trade-off, not a fix):** `indexing.wasb.stream_video=false` + `indexing.wasb.num_workers>1` ? parallel CPU decode keeps the L4 fed. Costs a full PNG extraction + disk. (Side benefit: also dodges the streaming resume-cache bug ? see companion issue.)
- **Batched/threaded streaming decode:** prefetch + decode frames on a small thread pool feeding the existing in-memory shim, so streaming isn't single-threaded. Keeps the no-PNG win without the disk cost.
- **True GPU decode (NVDEC):** decode on the L4 via `cv2.cudacodec.VideoReader` (needs a CUDA-built OpenCV), `PyNvVideoCodec`/`decord`-GPU, or an `ffmpeg -hwaccel cuda` pipe. Biggest win on long clips; biggest change (new dep + tensor-on-GPU handoff, must preserve the byte-identical frame contract the pipeline relies on).

Impact / severity

High for cost/throughput on the GPU-VM profile: you pay ~$0.70?0.85/hr for an L4 that's ~80% idle during detection because decode is the bottleneck. Fixing decode parallelism (or NVDEC) is the main lever to cut per-match wall-clock and make the L4 worth its cost.

Environment: GCP L4 VM (`g2-standard-4`), wasb env torch 1.11.0+cu113, OpenCV 4.13.0 (no cudacodec), `stream_video=true`, observed 2026-06-23.

11. user (2026-07-02T02:48:01.072Z)

```
271-    detector's sliding-window access pattern (peak O(window) frames, not O(video)).
272-
273-    The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
274:    ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
275-    in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
276-    start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
277-    boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
--
407-    sport: str = "badminton",
408-    limit: int = 0,
409-    batch_size: int = 8,
410:    num_workers: int = 4,
411-    log_every_batches: int = 50,
412-    cache_dir: "str | None" = None,
```

```
413-    flush_every: int = 1000,
--
428-      reader, which returns a PIL RGB image) over the DataLoader pass so every other line of
429-      ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
430-      to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
431:      ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
``num_workers=0``
432-      (the in-process shim + buffer can't cross worker processes).
433-
434-    ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
--
548-        )
549-        # Streaming forces single-process loading: the in-memory frame store + the
550-        # cv2.imread shim live in THIS process and can't be pickled to DataLoader workers.
551:        loader_workers = 0 if streaming else num_workers
552-        loader = DataLoader(
553:            ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
554-        )
555-        detector = build_detector(cfg)
556-
--
564-        pending = []
565-        logger.info(
566-            f"detecting on {len(remaining)} remaining windows "
567:            f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
568-            f"source={'video-stream' if streaming else 'frames-dir'})..."
569-        )
570-        det_f = open(det_jsonl, "a")
--
764-            sport,
765-            limit,
766-            batch_size=batch_size,
767:            num_workers=0,
768-            log_every_batches=log_every_batches,
769-            cache_dir=cache_dir,
770-            flush_every=flush_every,
--
857-        args.sport,
858-        args.limit,
859-        batch_size=args.batch_size,
860:        num_workers=args.num_workers,
861-        log_every_batches=args.log_every_batches,
862-        cache_dir=args.cache_dir,
863-        flush_every=args.flush_every,
```

12. user (2026-07-02T02:48:02.405Z)

```
58-    # decoded frame cache) are deleted automatically ? they are pure intermediates,
59-    # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60-    # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61:    keep_frames: bool = False
62-    # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of the
63-    # ~15×-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS, not
warns).
64-    # 0 = no extra floor (the headroom guard still applies).
--
95-    # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96-    # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates, regenerable
97-    # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98:    keep_frames: bool = False
99-    # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of the
100-     # ~15×-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS, not
warns).
```

```
101-    # 0 = no extra floor (the headroom guard still applies).
--
109-    #   None       = per-runner default ? the WSL runner keeps DISK extraction (unchanged
Windows
110-    #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111-    #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2 can't
seek).
112:    stream_video: Optional[bool] = None
113-
114-
115-class FusionConfig(BaseModel):
```

13. assistant (2026-07-02T02:48:10.045Z)

Issue #349 confirmed still present at wasb_infer.py:551. Interesting ? line 767 hardcodes `num_workers=0` in some wrapper. Let me see that context.

14. user (2026-07-02T02:48:11.256Z)

```
720                 state["pos"] = index
721             ok, frame = cap.read()
722             if not ok:
723                 raise RuntimeError(f"failed to decode frame {index} from {video_path}")
724             state["pos"] += 1
725             return frame
726
727         def release():
728             cap.release()
729
730         return frame_loader, count_hint, release
731
732
733     def run_video_streaming(
734         video_path: str,
735         weights: str,
736         out_csv: str,
737         sport: str = "badminton",
738         limit: int = 0,
739         batch_size: int = 8,
740         log_every_batches: int = 50,
741         cache_dir: "str | None" = None,
742         flush_every: int = 1000,
743         fresh: bool = False,
744         device: str = "cuda",
745     ) -> str:
746         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
747         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
748         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
749         ``run``'s docstring for why.
750         """
751         frame_loader, count_hint, release = make_video_frame_loader(video_path)
752         try:
753             if count_hint <= 0:
754                 raise SystemExit(
755                     f"could not determine frame count for {video_path}; cannot stream "
756                     f"(fall back to frame extraction with --frames_dir)"
757                 )
758             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
759             source_key = "video:" + osp.abspath(video_path)
760             return run(
761                 frame_list,
762                 weights,
763                 out_csv,
764                 sport,
```

```
765                 limit,
766                 batch_size=batch_size,
767                 num_workers=0,
768                 log_every_batches=log_every_batches,
769                 cache_dir=cache_dir,
770                 flush_every=flush_every,
771                 fresh=fresh,
772                 frame_loader=frame_loader,
773                 source_key=source_key,
774                 device=device,
775             )
776         finally:
777             release()
778
779
780     def main():
781         ap = argparse.ArgumentParser()
782         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
783         ap.add_argument(
784             "--video", help="video file; frames are extracted internally (in WSL)"
```

15. assistant (2026-07-02T02:48:18.675Z)

Now let me check issue #330 (MD5 dedup) and #331 (per-phase breakdown) implementation state.

16. user (2026-07-02T02:48:20.730Z)

```
Found 21 files
backend\utils\annotation_proxy.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\wasb_runner.py
backend\main.py
backend\api\uploads.py
backend\config\models.py
backend\worker\__main__.py
backend\utils\workspace.py
backend\storage\gdrive_api.py
backend\reporting\harvest.py
backend\pipeline\detectors\native_wasb_runner.py
backend\eval\regression.py
backend\eval\splits.py
backend\eval\golden_fixtures.py
backend\eval\golden_real_fixtures.py
backend\compute\base.py
backend\annotations\base.py
backend\annotations\file_provider.py
backend\reporting\spec.yaml
backend\utils\hashing.py
```

17. user (2026-07-02T02:48:23.870Z)

Measured on the live alpha (2-core CPU droplet), a real 411s (6.9 min) 1080p match ? 26 rallies

```
| Phase | Wall time | Bound by | Lever |
|------|-----------|----------|-------|
| Upload (7 chunks) | ~30 s | client upstream bandwidth | (network, not us) |
| Validation | ~87 s | **CPU** (frame decode/validators) | GPU/NVDEC, faster CPU |
| **Chunk extraction (6× ffmpeg re-encode + downscale)** | **~5.5 min** | **CPU** (software
libx264) | **GPU/NVENC**, more cores, **skip the downscale**, parallelize |
| Gemini upload + analysis | ~1.5 min | network (upload) + Gemini API (~15s/chunk, parallel) |
(fixed ? Google's side) |
| Reel stitch (trim+re-encode 10 clips ? concat ? watermark) | ~3 min | **CPU** (per-clip re-
encode) | **GPU/NVENC**; also async-compile (#329) so it doesn't 524 |
```

~65% of the wait is one thing: ffmpeg re-encoding on 2 cores (extraction + per-clip trim). The Gemini *analysis* is fast and external; the upload is client bandwidth.

Recommended levers (in impact order)

1. **GPU box (L4) with NVENC/NVDEC** ? collapses extraction + stitch from minutes to seconds; **also unlocks the owned WASB detector** (higher rally-detection quality) ? speed **and** quality. (Per ADR-012/013, GCP L4 / Cloud Run GPU.)
2. **Software (no new HW):** skip/relax the `ai_scale_width` chunk downscale (trade upload bytes for far less encode); parallelize extraction across cores; faster encode preset.
3. **Async `/api/compile`** (#329) so a slow stitch returns 202 + poll instead of a Cloudflare 524.

Baseline numbers above are a clean before/after reference once a GPU/optimizations land.

Ask (from the owner)

"If I upload the same video, I hope we do an MD5 checksum to reuse the Gemini segmentation."

Today

`video_id = compute_video_id(file)` **is** the MD5 of the file content, and a video's rallies are stored in `play_segments` keyed by `video_id`. But `/api/process` currently **re-runs the full segmentation** (Gemini) on every submit ? re-paying Gemini (paid) and re-waiting (~8 min on the CPU box) even for an already-segmented, byte-identical file. No reuse short-circuit found in `main.py` / `job_worker.py`.

Proposed

On `/api/process`, if `video_id` already has stored segments **for the same segmenter + relevant config**, **reuse** them and skip Gemini (return immediately with the existing rallies). Add an explicit **`force_resegment`** flag to override. The cache key already exists (MD5 `video_id`); this is mostly a short-circuit + the match condition.

Care

- Only reuse when the segmenter and segmentation-affecting config match (don't serve `gemini` segments for a `wasb_hybrid` request).
- Keep the existing "destroy-after-confirm" re-ingest semantics for an explicit re-segment.
- Big win: Gemini is paid + the CPU re-processing is the slow path; reuse makes re-opening a video instant and free.

18. user (2026-07-02T02:48:33.369Z)

```
backend\flywheel.py:15:This module is the PLUMBING ? it REUSES the real ``workspace`` /
``eval.experiment`` /
backend\flywheel.py:65:    source video. Returns the written URIs. Reuses
:func:`backend.storage.put` + the ``VideoWorkspace``
backend\infrastructure\database.py:233:        # so a new segmenter reuses this table by setting
its own `detector`/`stats`.
backend\worker\__main__.py:419:    Reuses the segmenter's `_resolve_runner` seam so the worker
and the live CLI build the
backend\reporting\spec.yaml:139:  # Re-processing an already-indexed video replaced its prior
segments (INSERT OR
backend\compute\cloud_run.py:9:     it is storage-backed and reuses the existing park/resume
primitives). Scale-to-zero (D7): the
backend\pipeline\segmenters\fusion_hybrid.py:3:Shuttle stays PRIMARY: this reuses an existing
trajectory substrate (WASB by default,
backend\pipeline\segmenters\fusion_hybrid.py:125:        # over the whole trajectory by
rally_gate (camera-agnostic); reused across windows.
backend\eval\distill_local.py:17:compare against the threshold baseline. Reuses classifier.py,
training.label_candidates,
backend\eval\calibrate_local.py:15:Reuses: TrackNetRunner windowing, rally_gate,
calibrate_wasb.load_golden_gts,
backend\pipeline\segmenters\trajectory_hybrid.py:325:        # segmenter adds net_crossings +
person-cue features. Computed once, reused for both
backend\pipeline\detectors\native_wasb_runner.py:21:name as the WSL runner) and reuses the
```

```
model-agnostic ``parse_trajectory_csv`` /
backend\pipeline\detectors\native_wasb_runner.py:48:# Reuse the model-agnostic helpers ?
trajectory shape + windowing are identical to WSL/TrackNet.
backend\pipeline\detectors\native_wasb_runner.py:215:        # M4: resolve the device. Cache the
probe so run_predict reuses it (no second subprocess).
backend\pipeline\detectors\native_wasb_runner.py:244:        # Content-derived key: same
filename + different bytes -> different key (no cache reuse).
backend\pipeline\detectors\wasb_runner.py:7:  4. reuses the (model-agnostic) trajectory->action-
window logic from tracknet_runner.
backend\pipeline\detectors\wasb_runner.py:27:# Reuse the model-agnostic helpers ? trajectory
shape + windowing are identical.
backend\pipeline\detectors\wasb_runner.py:137:        can never reuse each other's staged
frames, resumable cache, or CSV.
backend\eval\ablation.py:41:Pure + offline + GPU-free. Reuses the persisted golden feature
substrate (``fusion_golden.py``
backend\eval\ablation.py:284:    Reuses ``compare`` verbatim ? learned variants stay LOVO-
honest, the C1+C4 honest stats
backend\eval\export_wasb_segments.py:8:Reuses (no new windowing/metrics/cutting logic):
backend\eval\export_wasb_segments.py:259:    """Cut one padded clip per exported rally (reuses
the rally slicer)."""
backend\eval\fusion_golden.py:4:candidate window, by running `PersonDetector` over the ORIGINAL
video. It reuses the
backend\eval\fusion_golden.py:75:    # Net axis for the person two-sided split ? reuse
rally_gate's camera-agnostic estimate
backend\pipeline\detectors\court_gate.py:9:Pure (no GPU / cv2): point-in-polygon over the court
polygon (reuses
backend\eval\eval_partial_labels.py:15:It reuses everything from the existing eval stack (no new
windowing/metrics/gate logic):
backend\pipeline\detectors\wasb_infer.py:3:WASB-SBDT only ships *dataset* evaluation; this is a
thin wrapper that reuses its
backend\pipeline\detectors\wasb_infer.py:158:    field) stay compatible with a CUDA run ? but a
cpu/mps run will NOT reuse a cuda cache
backend\pipeline\detectors\wasb_infer.py:322:    Reuses ``_frame_id`` (digits-only parse) so the
index and the cached frame-id stay
backend\eval\experiment.py:640:    pinned ``classifier_model`` (you cannot train on an unlabeled
video). Reuses

[Showing results with pagination = limit: 30]
```

19. user (2026-07-02T02:48:34.365Z)

```
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api:
__init__.py
__pycache__
auth.py
job_worker.py
static
uploads.py

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/worker:
__init__.py
__main__.py
__pycache__
```

20. assistant (2026-07-02T02:48:40.345Z)

No `force_resegment` anywhere ? #330 unimplemented. Let me confirm the process path in job_worker.py and check ffmpeg subprocess handling.

21. user (2026-07-02T02:48:42.348Z)

```
29:# parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
43:def _get_executor() -> ThreadPoolExecutor:
58:    is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm contract.
```

```
60:    The wiring is additive: the production segmenter path never raises this today (zero
65:    def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
71:def _job_to_request(job: Dict[str, Any]):
81:        segmenter=job.get("segmenter"),
89:def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
103:    # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
140:def _maybe_record_job_cost(
182:def _maybe_run_flywheel(
205:def _drain_due_jobs(config: Any, db: Any) -> int:
230:def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
247:def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
```

22. user (2026-07-02T02:48:43.435Z)

```
backend\main.py:1912:        subprocess.run([sys.executable, doctor_path])
backend\worker\__main__.py:382:        out = subprocess.check_output(
backend\worker\__main__.py:606:    rc = subprocess.run(cmd).returncode
backend\pipeline\audio\hit_detector.py:53:    subprocess.run(
backend\utils\annotation_proxy.py:31:        out = subprocess.check_output(
backend\utils\annotation_proxy.py:81:        subprocess.run(cmd, check=True,
capture_output=True)
backend\pipeline\detectors\base.py:115:         return subprocess.run(
backend\utils\run_telemetry.py:69:        out = subprocess.run(
backend\utils\run_telemetry.py:123:         out = subprocess.run(
backend\tools\rally_slicer.py:117:        out = subprocess.run(
backend\tools\rally_slicer.py:185:            subprocess.run(cmd, capture_output=True,
check=True)
backend\pipeline\detectors\native_wasb_runner.py:129:        return subprocess.run(
backend\utils\video.py:18:            subprocess.check_output(
backend\utils\video.py:75:        subprocess.run(
backend\pipeline\validators\resolution_fps.py:15:        result = subprocess.run(
backend\pipeline\validators\format.py:43:            result = subprocess.run(cmd,
capture_output=True, text=True, check=True)
backend\pipeline\validators\pts_continuity.py:35:            result = subprocess.run(cmd,
capture_output=True, text=True, check=True)
backend\pipeline\stitcher\compiler.py:130:            result = subprocess.run(
backend\pipeline\stitcher\compiler.py:321:                subprocess.run(cmd,
capture_output=True, check=True)
backend\pipeline\stitcher\compiler.py:494:                subprocess.run(concat_cmd,
capture_output=True, check=True)
backend\pipeline\segmenters\gemini.py:166:                subprocess.run(
```

23. assistant (2026-07-02T02:48:49.965Z)

Now check which of these ffmpeg subprocess calls carry timeouts.

24. user (2026-07-02T02:48:51.687Z)

No matches found

25. user (2026-07-02T02:48:56.340Z)

```
331:        # generate() give-ups (per-attempt timeout / 503 bursts exhausting retries) are the
SAME
341:                f"{generate_fails}/{num_chunks} chunks FAILED generate (timeout/503 after
retries) ? "
```

26. user (2026-07-02T02:48:57.038Z)

No matches found

27. user (2026-07-02T02:48:57.890Z)

```
101-    """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.
102-
103-    Lives beside (not on) DetectorRunner: a future Linux-native runner must not
104-    inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
105:    and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
106:    ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
107-    """
108-
109-    cfg: Any
110-
--
112-        full = f"source {self.cfg.conda_sh} && {inner_cmd}"
113-        argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
114-        logger.debug("WSL exec: %s", full)
115-        return subprocess.run(
116:            argv, capture_output=True, text=True, timeout=(self.cfg.timeout_sec or None)
117-        )
118-
119-    # ---- cache hygiene helpers (shared by both WSL runners) ------------------
120-    def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
```

28. user (2026-07-02T02:49:06.591Z)

```
110             """
111             parsed = []
112             for raw_s, raw_e in segments:
113                 s = VideoCompiler._parse_time(raw_s)
114                 e = VideoCompiler._parse_time(raw_e)
115                 if e > s:
116                     parsed.append((s, e))
117             parsed.sort()
118             merged: List[Tuple[float, float]] = []
119             for s, e in parsed:
120                 if merged and s <= merged[-1][1]:  # overlaps/abuts previous -> extend it
121                     merged[-1] = (merged[-1][0], max(merged[-1][1], e))
122                 else:
123                     merged.append((s, e))
124             return merged
125
126         def _get_video_duration(self, video_path: str) -> float:
127             """Probes the video file to get its total duration in seconds.
128             Returns 0.0 if it cannot be determined."""
129             try:
130                 result = subprocess.run(
131                     [
132                         ffprobe_bin(),
133                         "-v",
134                         "error",
135                         "-show_entries",
136                         "format=duration",
137                         "-of",
138                         "default=noprint_wrappers=1:nokey=1",
139                         video_path,
140                     ],
141                     capture_output=True,
142                     text=True,
143                     check=True,
144                 )
145                 return float(result.stdout.strip())
146             except Exception as e:
147                 logger.warning(f"Could not determine video duration via ffprobe: {e}")
148                 return 0.0
149
150         def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
151             """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
```

```
152             and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
153             watermarking is disabled or has nothing to draw. The concat stage stays -c copy
154             because the mark is already baked into every clip identically."""
```

29. user (2026-07-02T02:49:07.589Z)

```
150
151                 cmd = [
152                     ffmpeg_bin(),
153                     "-y",
154                     "-ss",
155                     "0",
156                     "-t",
157                     str(debug_duration),
158                     "-i",
159                     video_path,
160                     "-c",
161                     "copy",
162                     temp_trimmed_path,
163                 ]
164                 logger.debug(f"Running command: {' '.join(cmd)}")
165                 try:
166                     subprocess.run(
167                         cmd,
168                         check=True,
169                         stdout=subprocess.DEVNULL,
170                         stderr=subprocess.DEVNULL,
171                     )
172                     logger.info(
173                         f"Trimmed video successfully created at: {temp_trimmed_path}"
174                     )
175                     video_to_upload = temp_trimmed_path
176                     is_trimmed = True
177                 except Exception as e:
178                     logger.warning(
179                         f"Failed to trim video using ffmpeg: {e}. Falling back to full
video."
180                     )
181
182             # Helper to clean up local temp files
183             def cleanup_temp_files():
184                 if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
185                     try:
186                         os.remove(temp_trimmed_path)
187                     except Exception:
188                         pass
189
```

30. user (2026-07-02T02:49:15.584Z)

```
1       import logging
2       import subprocess
3       from typing import Optional
4
5       from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
6           ffmpeg_bin,
7           ffprobe_bin,
8       )
9
10      logger = logging.getLogger(__name__)
11
12
13      def get_video_duration(video_path: str) -> float:
14          """Returns the exact duration of a video file in seconds using ffprobe.
15          Returns 0.0 on failure."""
```

```
        try:
            duration_output = (
                subprocess.check_output(
                    [
                        ffprobe_bin(),
                        "-v",
                        "error",
                        "-show_entries",
                        "format=duration",
                        "-of",
                        "default=noprint_wrappers=1:nokey=1",
                        video_path,
                    ]
                )
                .decode()
                .strip()
            )
            return float(duration_output)
        except Exception as e:
            logger.error(f"Could not detect video duration for {video_path}: {e}")
            return 0.0


    def extract_video_chunk(
        video_path: str,
        start_time: float,
        duration: float,
        output_path: str,
        reencode: bool = False,
        scale_width: Optional[int] = None,
    ) -> bool:
        """Extracts a chunk from a video file.

        Args:
            reencode: If True, re-encodes with libx264 (slower but frame-accurate).
                      If False, uses stream-copy (fast but cuts at nearest keyframe).
                      Use True for short clips where keyframe drift matters.
            scale_width: If set (requires reencode), downscale to this width (height auto,
                      even). Useful for high-res sources headed to an API that analyses at
                      low res anyway ? shrinks uploads with no analysis loss.
        """
        cmd = [
            ffmpeg_bin(),
            "-y",
            "-ss",
            str(start_time),
            "-t",
            str(duration),
            "-i",
            video_path,
        ]
        if reencode:
            if scale_width:
                cmd += ["-vf", f"scale={int(scale_width)}:-2"]
            cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
        else:
            cmd += ["-c", "copy"]
        cmd.append(output_path)
        try:
            subprocess.run(
                cmd, check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
            )
            return True
        except Exception as e:
            logger.error(
```

```
81                  f"Failed to extract chunk {start_time}s - {start_time + duration}s: {e}"
82              )
83              return False
84
```

31. user (2026-07-02T02:49:28.214Z)

```
25      logger = logging.getLogger(__name__)
26
27      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
28      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
29      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
30      # already parallelize their AI handoff internally via their own pools.
31      #
32      # ? O2 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
33      # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
34      # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
35      #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
36      #   - detector timeout_sec kills hung WSL/GPU calls
37      #   - Cloud Run poll has a max_wait_sec bound
38      # A future slice should add a per-job wall-clock timeout (the executor itself
39      # cannot enforce timeouts on the Thread).
40      _job_executor: Optional[ThreadPoolExecutor] = None
41
42
43      def _get_executor() -> ThreadPoolExecutor:
44          """Lazy module-global executor; tests monkeypatch this for inline execution."""
45          global _job_executor
46          if _job_executor is None:
47              _job_executor = ThreadPoolExecutor(
48                  max_workers=1, thread_name_prefix="jobworker"
49              )
50          return _job_executor
51
52
53      class JobWaiting(Exception):
54          """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
55          WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
56          NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
57          releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
58          is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
59
60          The wiring is additive: the production segmenter path never raises this today (zero
61          behaviour change), so a job runs exactly as before. The hook is what a later slice
62          (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
63          """
64
65          def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
66              super().__init__(f"job parked for {delay_sec:.0f}s")
67              self.delay_sec = max(0.0, float(delay_sec))
68              self.progress = progress
69
```

32. user (2026-07-02T02:49:29.704Z)

```
backend\config\models.py:61:    keep_frames: bool = False
backend\config\models.py:98:    keep_frames: bool = False
backend\pipeline\detectors\native_wasb_runner.py:82:    keep_frames: bool = (
backend\pipeline\detectors\native_wasb_runner.py:110:            keep_frames=w.keep_frames,
backend\pipeline\detectors\native_wasb_runner.py:315:        if not self.cfg.keep_frames and
frames_dir:
backend\pipeline\detectors\native_wasb_runner.py:318:                "(set
```

```
indexing.wasb.keep_frames=true to retain).",
backend\pipeline\detectors\tracknet_runner.py:48:    keep_frames: bool = False  # retain staged
video after success (debug only)
backend\pipeline\detectors\tracknet_runner.py:66:            keep_frames=tn.keep_frames,
backend\pipeline\detectors\tracknet_runner.py:225:        if self.cfg.stage_in_wsl and not
self.cfg.keep_frames:
backend\pipeline\detectors\tracknet_runner.py:228:                "(set
indexing.tracknet.keep_frames=true to retain).",
backend\pipeline\detectors\wasb_runner.py:51:    keep_frames: bool = (
backend\pipeline\detectors\wasb_runner.py:75:            keep_frames=w.keep_frames,
backend\pipeline\detectors\wasb_runner.py:210:        if not self.cfg.keep_frames:
backend\pipeline\detectors\wasb_runner.py:216:                "(set
indexing.wasb.keep_frames=true to retain).",
```

33. user (2026-07-02T02:49:46.534Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-2-
backend\infrastructure\execution_policy.py-3-Turns an unbounded blocking call (the GX010125
Gemini hang) into a bounded one:
backend\infrastructure\execution_policy.py:4:- ``run_bounded`` ? a HARD per-attempt timeout
(``op_timeout_sec``, the hang-killer) + retry-with-backoff
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-5-  for FAILURES (an exception) *and* HANGS
(no return in time). A hung attempt's worker thread is a daemon,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-6-  so it is abandoned, not joined ? it never
blocks the process.
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-40-
backend\infrastructure\execution_policy.py-41-class PolicyTimeout(TimeoutError):
backend\infrastructure\execution_policy.py:42:    """An op hung past ``op_timeout_sec`` or never
became ready within ``max_wait_sec``."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-43-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-44-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-45-@dataclass(frozen=True)
backend\infrastructure\execution_policy.py:46:class ExecutionPolicy:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-47-    """Declarative resilience knobs,
attachable per job/op (JSON-serialisable; see EXECUTION_POLICY.md)."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-48-
backend\infrastructure\execution_policy.py-49-    mode: WaitMode = WaitMode.WAIT_AND_RESUME
backend\infrastructure\execution_policy.py-50-    poll_every_n_sec: float = 10.0
backend\infrastructure\execution_policy.py:51:    op_timeout_sec: float = (
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-52-        120.0  # HARD per-attempt deadline
? the hang-killer that was missing
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-53-    )
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-114-
backend\infrastructure\execution_policy.py-115-        threading.Thread(target=_runner,
name=f"bounded:{label}", daemon=True).start()
backend\infrastructure\execution_policy.py:116:        if
done.wait(timeout=policy.op_timeout_sec):
backend\infrastructure\execution_policy.py-117-            if "exc" in box:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-118-                last_exc = box["exc"]
--
```

```
backend\infrastructure\execution_policy.py-132-        else:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-133-            last_exc = PolicyTimeout(
backend\infrastructure\execution_policy.py:134:                f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-135-            )
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-136-            log.warning(
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-139-                attempt + 1,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-140-                attempts,
backend\infrastructure\execution_policy.py:141:                policy.op_timeout_sec,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py-142-            )
backend\infrastructure\execution_policy.py-143-        if attempt < attempts - 1:
```

34. user (2026-07-02T02:49:47.343Z)

```
"""Cache-hygiene + retention sweep for the GPU highlight pipeline.

Scans the regenerable intermediates the pipeline leaves behind ? in the WSL stage
dir (``~/clips``), the Windows ``output/`` tree, and the upload landing zone
(``security.upload_dir``, default ``output/uploads``) ? classifies each entry as
**purge** (regenerable: frame caches, staged video copies, handoff clips, temp dirs;
and, past a retention window, uploaded source videos) or **keep** (durable: trajectory
CSVs, the SQLite DB, final reels) and reports the reclaimable space.

**Retention sweep (T11 ? pre-alpha-exposure):** uploaded source videos in the upload
dir are a *privacy*-driven purge, not a *disk*-driven one ? once a tester's footage is
older than ``--uploads-retention-days`` (default 7) it is swept, along with the orphaned
``.partial-<id>`` temp files aborted uploads leave behind. Reels / DB / CSVs stay durable
per the cache-hygiene policy (the owner's call, 2026-06-21).

DRY-RUN BY DEFAULT ? it only *shows* what it would delete. Pass ``--apply`` to delete.

    python -m tools.cleanup_caches                      # report only (safe)
    python -m tools.cleanup_caches --apply              # delete purge candidates
    python -m tools.cleanup_caches --keep-video 89605e80ed01   # protect one video's cache
    python -m tools.cleanup_caches --older-than 7       # only purge regenerable caches >7d old
    python -m tools.cleanup_caches --uploads-retention-days 3   # sweep uploads >3d old
    python -m tools.cleanup_caches --include-wasbcache  # also drop detector resume caches
    python -m tools.cleanup_caches --json               # machine-readable plan (for a cron/job
wrapper)

Pairs with the pipeline's own self-cleaning (indexing.{wasb,tracknet}.keep_frames=false,
which deletes frames after a SUCCESSFUL run); this tool reclaims what failed/old runs
and pre-self-cleaning videos left behind, and enforces the upload retention window before
non-owner (alpha) exposure. Stdlib only; safe to run anywhere (the WSL scan is skipped
automatically when ``wsl`` is unavailable). ``--uploads-dir`` must match ``security.upload_dir``
(its default matches the config default, ``output/uploads``).
"""

from __future__ import annotations

import argparse
import os
import shutil
import subprocess
import sys
from dataclasses import dataclass
from typing import Dict, List, Optional, Tuple
```

---- classification (pure; unit-tested) ----------

(category, regenerable, default_purge). default_purge=True ? deleted unless a flag protects it. wasbcache is regenerable but default_purge=False (it enables resume and is tiny) ? opt in with --include-wasbcache.

```
_KEEP = ("keep", False, False)


def classify(name: str, is_dir: bool, location: str) -> Tuple[str, bool, bool]:
    """Classify a cache entry by name. Returns (category, regenerable, default_purge)."""
    low = name.lower()
    if location == "uploads":
        # The upload landing zone (security.upload_dir). Uploaded source videos and the
        # orphaned `.partial-<id>` temp files aborted uploads leave behind are both swept
        # past the retention window (default_purge=True; held by --uploads-retention-days
        # in plan_deletions). Anything else here ? e.g. a future per-user subdir (PR-3) ? is
        # left untouched, since we don't recurse into unknown upload trees.
```

35. user (2026-07-02T02:49:58.185Z)

```
backend\providers\gemini.py:15:    run_bounded,
backend\providers\gemini.py:233:            response = run_bounded(
backend\infrastructure\execution_policy.py:4:- ``run_bounded`` ? a HARD per-attempt timeout
(``op_timeout_sec``, the hang-killer) + retry-with-backoff
backend\infrastructure\execution_policy.py:83:def run_bounded(
```

36. user (2026-07-02T02:49:59.510Z)

```
README.md:171:*   **Manual sweep (dry-run-first):** `tools/cleanup_caches.py` reports and (with
`--apply`) reclaims what failed/old runs left behind, across both the WSL stage dir and
`output/`:
README.md:173:    python -m tools.cleanup_caches                  # report what's reclaimable
(safe; deletes nothing)
README.md:174:    python -m tools.cleanup_caches --apply          # delete the regenerable
caches
README.md:175:    python -m tools.cleanup_caches --keep-video GX010125   # protect one video's
cache (id or filename stem)
README.md:176:    python -m tools.cleanup_caches --older-than 7   # only purge entries older
than 7 days
README.md:249:[Omitted long matching line]
README.md:443:  cleanup_caches.py       # Dry-run-first cache GC (WSL ~/clips + output/); see
"Cache Hygiene" above
docs\PER_VIDEO_WORKSPACE.md:67:- **REUSE** `backend/storage/`
(`parse_uri`/`fetch`/`put`/`list_uris`) for all durable read/write; `compute_video_id` for the
key; the existing `keep_frames`/`min_free_gb` cache-hygiene; `tools/cleanup_caches.py` (retarget
to `tmp/`).
docs\PER_VIDEO_WORKSPACE.md:87:| P5 | **Launch Report** + flip default ON + retarget
`tools/cleanup_caches.py` + docs | P2,P3,P4 | ? owner | ? | ? |
backend\pipeline\detectors\base.py:162:            f"Free space (e.g. python -m
tools.cleanup_caches) and retry."
tests\test_cleanup_caches.py:8:from tools.cleanup_caches import Entry, classify, plan_deletions
tests\test_cleanup_caches.py:198:    from tools.cleanup_caches import _by_category
tests\test_cleanup_caches.py:208:    from tools.cleanup_caches import scan_uploads, scan_windows
docs\CODE_MAP.md:30:| **Ops / maintenance tool** (not part of the served app ? cache GC, batch
admin) | `tools/` | Run as `python -m tools.<name>` (it's an importable package). e.g.
`tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested. |
docs\CODE_MAP.md:195:[Omitted long matching line]
docs\CODE_MAP.md:317:| Reclaim disk / clean caches | `python -m tools.cleanup_caches` (dry-run)
then `--apply`; runs also self-clean via `indexing.{wasb,tracknet}.keep_frames=false` (default)
+ `min_free_gb` guard; tool `@tools/cleanup_caches.py` |
docs\archives\PER_VIDEO_OUTPUT_LAYOUT-SUPERSEDED-2026-06-21.md:67:- **L3 ? cache-hygiene +
cleanup_caches awareness** of the new layout (per-video purge).
```

37. user (2026-07-02T02:50:15.818Z)

```
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy:
cloudrun
digitalocean
gcp

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy/cloudrun:
Dockerfile
README.md
cloudbuild.yaml

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy/digitalocean:
README.md
bootstrap.sh
gpu_setup.sh
mount_scratch.sh

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy/gcp:
README.md
buckets.env
create_gpu_vm.sh
nightly_regression
ops-agent-startup.sh
provision.sh
teardown.sh

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/deploy/gcp/nightly_regression:
Dockerfile.launcher
README.md
cloudbuild.launcher.yaml
create_nightly_bucket.sh
launch.sh
register_scheduler.sh
run_on_vm.sh
setup_email.sh
```

38. user (2026-07-02T02:50:16.701Z)

```
backend\pipeline\segmenters\gemini.py:17:from backend.utils.video import extract_video_chunk,
get_video_duration
backend\pipeline\segmenters\gemini.py:232:            success = extract_video_chunk(
backend\eval\gemini_refine.py:11:sports.badminton prompt (sport_registry),
utils.video.extract_video_chunk,
backend\eval\gemini_refine.py:35:from backend.utils.video import extract_video_chunk,
get_video_duration
backend\eval\gemini_refine.py:93:    if not extract_video_chunk(
backend\eval\gemini_refine.py:245:        if not extract_video_chunk(
backend\pipeline\segmenters\trajectory_hybrid.py:35:from backend.utils.video import
extract_video_chunk, get_video_duration
backend\pipeline\segmenters\trajectory_hybrid.py:395:                if not extract_video_chunk(
backend\pipeline\segmenters\yolo_hybrid.py:30:from backend.utils.video import
extract_video_chunk, get_video_duration
backend\pipeline\segmenters\yolo_hybrid.py:155:                success =
extract_video_chunk(video_path, cs, actual_dur, chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py:205:                success = extract_video_chunk(
backend\pipeline\segmenters\yolo_hybrid.py:341:            success = extract_video_chunk(
backend\utils\video.py:39:def extract_video_chunk(
```

39. user (2026-07-02T02:50:17.578Z)

```
590             manifest_path,
591             "--out",
592             out_dir,
```

```
            "--version",
            args.version,
        ]
        if args.floor:
            floor_local = (
                storage.fetch(
                    args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
                )
                if "://" in args.floor
                else args.floor
            )
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
```

40. user (2026-07-02T02:50:25.858Z)

```
deploy\gcp\nightly_regression\launch.sh:4:# and kicks the VM off. Driver-agnostic: call it from
Cloud Scheduler?Cloud Run job, a cron on any
deploy\gcp\nightly_regression\create_nightly_bucket.sh:13:#   RALLY_NIGHTLY_TTL_DAYS=14 bash
.../create_nightly_bucket.sh           # custom retention
deploy\gcp\nightly_regression\register_scheduler.sh:3:#   Cloud Scheduler (cron)  ?  Cloud Run
JOB (the launcher image)  ?  launch.sh
deploy\gcp\nightly_regression\register_scheduler.sh:13:#   RALLY_CRON='0 3 * * *'
RALLY_TZ='Asia/Kolkata' bash .../register_scheduler.sh
deploy\gcp\nightly_regression\register_scheduler.sh:24:CRON="${RALLY_CRON:-0 3 * * *}"
# 03:00 daily
deploy\gcp\nightly_regression\register_scheduler.sh:65:  --schedule="$CRON" --time-zone="$TZ" \
deploy\gcp\nightly_regression\register_scheduler.sh:71:? Scheduled '$SCHED' ($CRON $TZ) ? Cloud
Run job '$JOB' ? launch.sh ? ephemeral GPU VM (SA IAM granted above).
deploy\gcp\nightly_regression\register_scheduler.sh:72:   ? START A RUN NOW (same as the nightly
cron does):
deploy\gcp\nightly_regression\README.md:21:Cloud Scheduler (cron, default 03:00 IST)
deploy\gcp\nightly_regression\README.md:45:| **Frequency / time** | `0 3 * * *` @ `Asia/Kolkata`
| Cloud Scheduler cron. Live: `gcloud scheduler jobs update http $S --location $R --schedule "0
4 * * *" --time-zone "Asia/Kolkata"` (or re-register with `RALLY_CRON`/`RALLY_TZ`) |
deploy\gcp\nightly_regression\README.md:55:| **Report retention** | 30 days | the nightly
bucket's lifecycle TTL ? `RALLY_NIGHTLY_TTL_DAYS` then re-run `create_nightly_bucket.sh` |
deploy\gcp\nightly_regression\README.md:96:## Run on demand (same path the cron uses)
```

41. user (2026-07-02T02:50:27.354Z)

Context

```
The T11 **upload retention sweep** landed in #299 (merged, `cb7394b`): `tools/cleanup_caches.py`
was extended to purge ? past a retention window ? uploaded source videos and orphaned
`.partial-<id>` files in `security.upload_dir` (default `output/uploads`), alongside the
existing regenerable-cache cleanup. It is **dry-run by default**, keeps reels/DB/CSVs durable,
and emits a machine-readable plan via `--json`.

That PR delivered the **tool**. What it did **not** do is run it **unattended**: today an
```

operator must manually run `python -m tools.cleanup_caches --apply` on the box. Before/while non-owner (alpha) users upload footage, the retention window should be enforced automatically without a human in the loop.

Goal

Wire the existing sweep into an **automated scheduled job** so stale uploads are purged on the owner-set retention window (uploads **7d**; reels kept durable) without manual intervention, with loud structured logging captured to observability.

Do not re-implement the sweep ? schedule the existing tool. The `--json` output (`{windows, uploads_dir, reclaimable_bytes, purge[], by_category[]}`) is already shaped for a wrapper/log sink.

What's needed

- A scheduler appropriate to the deployment:
  - **Single-box alpha** (engine on one host): a host **cron** or **systemd-timer** entry that runs the sweep daily.
  - **Cloud Run serving path**: a **Cloud Scheduler ? job** invocation (or a sidecar), consistent with `COMPUTE_DECOUPLED_SERVING/`.
- The scheduled invocation runs `--apply` with `--uploads-retention-days 7` and `--uploads-dir` matching `security.upload_dir` (consider resolving from config rather than hardcoding the path).
- **Default-OFF / opt-in** so the single-user local install is never surprised by an unattended deletion job.
- **Observability**: capture the `--json` plan + the per-item `PURGED/FAILED` lines to logs each run (counts + bytes freed + the active windows), per the launch quality bar (loud, structured, correlation id).
- **Document** it in the deploy runbook (engine `deploy/` and/or `khelsutra/deploy/DEPLOY_ALPHA.md`).

Design considerations / open questions

- **In-flight safety:** the retention window already protects recent `.partial-<id>` files (an in-flight chunked upload < window is never swept). Confirm the cadence + window leave a comfortable margin over the max upload duration.
- **Active-job safety:** a *finished* upload may still be referenced by an in-progress `/api/process` job. Decide whether the scheduled apply should additionally skip uploads referenced by a non-terminal row in the `jobs` table (defense-in-depth beyond the time window). The window alone is probably sufficient for 7d, but worth a deliberate call.
- **Per-user subdirs (PR-3):** the sweep currently leaves an unknown upload **subdir** untouched (`classify` returns `upload subdir` = keep). When identity scoping lands, the job/tool will need to recurse per-user.
- **Apply guard:** since the scheduled job runs `--apply`, gate it behind an explicit config flag + keep the conservative window so a misconfiguration can't nuke fresh uploads.

Acceptance criteria

42. user (2026-07-02T02:50:50.965Z)

```
115     class NativeWasbRunner(DetectorRunner):
116         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
117
118         def __init__(self, cfg: NativeWasbConfig):
119             self.cfg = cfg
120             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
121             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
122             self._cuda_available: Optional[bool] = None
123
124         # --- low-level native exec (the single place tests patch)
```

```
-------------------------
125         def _run(
126             self, argv: List[str], cwd: Optional[str] = None
127         ) -> subprocess.CompletedProcess:
128             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
129             return subprocess.run(
130                 argv,
131                 cwd=cwd,
132                 capture_output=True,
133                 text=True,
134                 timeout=(self.cfg.timeout_sec or None),
135             )
136 
137         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) ------
138         def _probe_cuda(self) -> bool:
139             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
140             (missing python / timeout / import error) is treated as 'no CUDA'."""
141             try:
142                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
143             except (FileNotFoundError, subprocess.TimeoutExpired):
144                 return False
```

43. user (2026-07-02T02:50:51.907Z)

```
1       """Canonical file-identity hashing for the indexer.
2 
3       `video_id` is the MD5 of the *whole* video file. Historically each entrypoint
4       (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
`scripts/run_process_and_stitch.py`)
5       defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
6       The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
7       any future change to one copy would silently diverge the DB key.
8 
9       This module is the single source of truth. Import `compute_video_id` everywhere a
10      video needs to be keyed.
11      """
12 
13      from __future__ import annotations
14 
15      import hashlib
16 
17      # 64 KB matches the value used by the live pipeline (backend/main.py) before this
18      # was centralized. The chunk size does not affect the resulting digest ? it only
19      # bounds memory use while streaming large files.
20      _CHUNK_SIZE = 65536
21 
22 
23      def compute_video_id(filepath: str) -> str:
24          """Return the MD5 hex digest of the whole file ? the canonical video_id.
25 
26          Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27          into memory at once.
28          """
29          hasher = hashlib.md5()
30          with open(filepath, "rb") as f:
31              for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
32                  hasher.update(chunk)
33          return hasher.hexdigest()
34 
```

44. user (2026-07-02T02:50:53.462Z)

```
docs\COMMERCIALIZATION.md:107:  (royalty-free)**. NVENC AV1 HW-encode needs **Ada-class GPUs
```

(L4/L40)**; Ampere/Turing
docs\DECISION_SNAPSHOT_REGRESSION.md:17:[Omitted long matching line]
docs\DECISION_SNAPSHOT_REGRESSION.md:30:| **D1** | **Snapshot the DECISION output, never the encoded reel.** Freeze the rally segment list + the stitch/concat plan; never MD5 the reel `.mp4`. | Encoded mp4 bytes are non-deterministic (ffmpeg version string, x264 threading, NVENC, mux timestamps) ? a reel-MD5 false-alarms on every ffmpeg bump/machine. The repo's own `nightly_reelcount.py` already asserts count+metrics, **not** bytes [verified]. | **LOCKED** (2026-06-25) |
docs\DESKTOP_APP_PLAN.md:45:- **WASB on a discrete laptop GPU is acceptable** `[verified, measured]`: on the owner's **RTX 2070 Super Max-Q (8 GB)**, WASB inference is the pipeline driver at **~3.2?3.4× real-time** (?3.4 GPU-min per footage-min, ~0.055 s/frame), peak VRAM ~5.7?6.0 GB; proxy (NVENC) ~0.2× RT; 4K stitch (CPU libx264) ~16 s/rally; end-to-end ?5× clip length. Source: `current-state.md` `COST_SCALING.md` checkpoint (6 Boxhill clips, 2026-06-18).
docs\DESKTOP_APP_PLAN.md:64:- **ffmpeg licensing (a genuine landmine):** ffmpeg is LGPL/GPL. The current stitch path uses **libx264 (GPL)**. Redistributing that inside an installer would impose GPL on the bundle. **Mitigation (D4):** ship an **LGPL ffmpeg build** using **openh264 (BSD)** or the OS hardware encoder (NVENC / Apple **VideoToolbox** / Linux **VAAPI**) ? the desktop app should prefer hardware H.264 anyway (faster, no CPU stitch tax).
docs\COMPUTE_DECOUPLED_SERVING\README.md:43:[Omitted long matching line]
docs\COMPUTE_DECOUPLED_SERVING\README.md:142:8. ? **Codec ? D16:** **libx264 ENCODE** (protect parity) + **NVDEC DECODE-only** (speed, no output-byte change); NVENC-encode deferred behind a config knob.
docs\archives\decisions\DECISIONS.md:246:4. **Emit royalty-free output.** Standardize highlight output on **AV1 (+ Opus)** to avoid AVC/HEVC encode-side patent exposure; provision an **Ada-class GPU (L4/L40)** for hardware AV1 (NVENC), falling back to **SVT-AV1 (CPU, BSD-3-Clause-Clear)** on T4/A10/A100.
docs\archives\MONETIZATION_AUDIT.md:96: - **GPU acceleration:** NVENC **AV1 encode requires Ada-class** GPUs (RTX 40, **L4 / L40**). Ampere (A10/A100) and Turing (T4) can *decode* AV1 but **cannot HW-encode** it ? use **SVT-AV1 (CPU)** on those, while H.264/HEVC encode stays HW-accelerated everywhere.
docs\archives\MONETIZATION_AUDIT.md:122:- ffmpeg legal: `ffmpeg.org/legal.html`. Codec pools: Via LA (`via-la.com`), Wikipedia H.264 / HEVC patent-licensing. NVIDIA NVENC matrix: `developer.nvidia.com/video-encode-and-decode-gpu-support-matrix`. SVT-AV1: AOMedia repo (GitLab).
docs\PACKAGING_PLAN.md:72:- **ffmpeg:** route **every** `ffmpeg`/`ffprobe` call through **one resolver** (config path ? env ? `shutil.which` ? fail-loud); first-run setup (extend `scripts/doctor.py`, which already prints the winget command) detects-and-offers `winget install Gyan.FFmpeg`. An LGPL ffmpeg bundle (BtbN, openh264/NVENC) is a documented fallback that also requires swapping the `libx264` default ? tracked, not v1.
docs\archives\past_projects\DECOUPLED_COMPUTE\02-COMPUTE-MAP-AND-CONTRACT.md:17:| **`ffmpeg` chunking / stitching** | CPU (`libx264`) | **LOW.** No nvenc/driver-codec dependency (a portability win) ? `apt-get install ffmpeg` is enough. |
docs\archives\past_projects\DECOUPLED_COMPUTE\02-COMPUTE-MAP-AND-CONTRACT.md:34:CPU-first; *"the gap is narrow."* Only GPU touchpoint is **opt-in `nvenc`** (`--encoder nvenc`, commit #28)
docs\archives\past_projects\DECOUPLED_COMPUTE\01-EXISTING-PLANS-AND-GAP.md:95:gap is narrow"*: its only GPU touchpoint is opt-in `nvenc` (defaults to CPU `libx264`); it shares the same
docs\archives\past_projects\DECOUPLED_COMPUTE\06-RISKS-AND-OPEN-QUESTIONS.md:34:5. **Data-contract break via re-encode.** MD5 keys the whole 3-repo join; the collector's `nvenc`/`libx264`

[Showing results with pagination = limit: 15]

45. user (2026-07-02T02:51:01.871Z)

No matches found

46. user (2026-07-02T02:51:04.305Z)

349   WASB native: decode is CPU-bound & single-threaded in streaming mode ? L4 GPU ~80% idle (no NVDEC/parallel decode)
331   [perf] Hosted processing is CPU-bound (ffmpeg/cv2) ? measured per-phase breakdown + levers (GPU/NVENC, parallel extraction, async compile)
330   [optimization] MD5 segmentation reuse ? skip Gemini re-segmentation when the same video (video_id) is re-uploaded
301   T11 follow-up: run the upload retention sweep as an automated scheduled job (hosted

alpha)

47. user (2026-07-02T02:51:27.140Z)

```
4:import tempfile
419:        with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
```

48. user (2026-07-02T02:51:28.447Z)

```
388     refactor: migrate async-job plumbing (_get_executor/_drain_due_jobs/_arm_wake_timer) out
of main.py
384     ci: single-source the shared offline guard lanes + coverage floor via a workflow_call
reusable
349     WASB native: decode is CPU-bound & single-threaded in streaming mode ? L4 GPU ~80% idle
(no NVDEC/parallel decode)
340     [cost] /api/process re-runs the paid segmenter for an already-processed video ?
refresh/"Try again" re-bills Gemini
332     [safety] Content-safety pre-flight litmus: flag non-sports / NSFW video BEFORE sending
it to Gemini
331     [perf] Hosted processing is CPU-bound (ffmpeg/cv2) ? measured per-phase breakdown +
levers (GPU/NVENC, parallel extraction, async compile)
330     [optimization] MD5 segmentation reuse ? skip Gemini re-segmentation when the same video
(video_id) is re-uploaded
328     Surface the per-video runlog / execution trace to the user (endpoint + UI), not just
server logs
327     Allow gs:// / s3:// / gdrive-api:// ingest on an exposed (jailed) box ? scheme-exempt
recognized storage backends from the path-jail
326     Hosted-app observability: request tracing + replay for debugging production CUJ failures
301     T11 follow-up: run the upload retention sweep as an automated scheduled job (hosted
alpha)
175     Shared cloud container (OneDrive) for GPU-extractor golden data ? sync GPU box ? dev
sessions
172     Calibrate the ship-gate F1@tIoU target (currently provisional ? NOT owner-ratified)
```

49. user (2026-07-02T02:51:39.523Z)

```
670:    Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is validated
as a bare
733:    Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a local
file from
1355:@app.post("/api/process", status_code=202)
1411:    if not request.app.state.db.create_job(
1598:    Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip + concat +
watermark)
```

50. user (2026-07-02T02:51:45.000Z)

```
1355    @app.post("/api/process", status_code=202)
1356    def process_video(req: ProcessRequest, request: Request):
1357        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
1358
1359        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
1360        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
1361        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
1362        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
1363        """
1364        # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
1365        # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
1366        # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
1367        try:
1368            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
```

```
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

        allowed_root = getattr(
            getattr(request.app.state.config, "security", None), "allowed_video_root", None
        )
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
            # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
            input_ref = storage.resolve_input_ref(
                req.video_path, getattr(request.app.state.config, "storage", None)
            )
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
        # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
        # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
        # rejects a non-local URI as out-of-root.
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
            raise HTTPException(
                status_code=404, detail=f"Video file not found at '{req.video_path}'"
            )
        # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
        # if this box can't hold that copy. Best-effort ? an unstattable remote
(network/SDK) or an
        # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
        # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
        if input_ref.backend != "local":
            need = _remote_input_size_bytes(
                input_ref, getattr(request.app.state.config, "storage", None)
            )
            require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
            )

        job_id = uuid.uuid4().hex
        if not request.app.state.db.create_job(
            job_id,
            req.video_path,
            req.segmenter,
            req.player_pool,
            req.max_frames,
            owner_id=owner_id,
            locale=req.locale,
            compute=req.compute,
        ):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(
```

```
1423            _drain_due_jobs, request.app.state.config, request.app.state.db
1424        )
1425        return JSONResponse(
1426            status_code=202,
1427            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
1428        )
1429
1430
1431    @app.get("/api/jobs/{job_id}/cost")
1432    def get_job_cost(job_id: str, request: Request):
1433        """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
1434        GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
```

51. user (2026-07-02T02:51:57.022Z)

```
No matches found
```

52. user (2026-07-02T02:51:58.285Z)

```
backend\main.py:771:            src_ref = storage.resolve_input_ref(req.uploaded_path,
storage_settings)
backend\main.py:783:            src_ref = storage.resolve_input_ref(req.source_uri,
storage_settings)
backend\main.py:786:        # A remote at-rest source is fetched to scratch to hash it (D3) ?
reserve that scratch (P2).
backend\main.py:1025:        input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
backend\main.py:1165:    # byte-identical to today); a bucket/Drive URI is fetched to scratch
first. The original
backend\main.py:1379:        input_ref = storage.resolve_input_ref(
backend\main.py:1394:    # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the
worker; fail fast with 507
backend\main.py:1681:        # a LOCAL file the stitcher can read ? identity for a local path,
fetch-to-scratch for a URI (the
backend\main.py:2044:        # M2: bare/local path = identity (today's behaviour); a
bucket/Drive URI is fetched to scratch.
backend\main.py:2049:            input_ref = storage.resolve_input_ref(args.video_path,
storage_cfg)
backend\storage\base.py:19:    """Backend-agnostic object storage. ``fetch_to_local`` + ``put``
are the two the
backend\storage\base.py:23:    def fetch_to_local(
backend\storage\base.py:61:        ``NotImplementedError`` as expected and fall back to
``fetch_to_local``."""
backend\storage\__init__.py:35:    "resolve_input_ref",
backend\storage\__init__.py:48:def resolve_input_ref(raw: str, storage_config: Any = None) ->
StorageRef:
backend\storage\__init__.py:68:    ``resolve_input_ref``)."""
backend\storage\__init__.py:69:    return resolve_input_ref(raw, storage_config).backend ==
"local"
backend\storage\__init__.py:84:    ref = resolve_input_ref(raw, sc)
backend\storage\__init__.py:90:    return fetch(ref.uri, os.path.join(scratch, ref.name or
"in.mp4"), config=sc)
backend\storage\__init__.py:151:    return get_storage(ref.backend,
config=config).fetch_to_local(
backend\storage\s3.py:47:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
backend\storage\s3.py:49:            raise ValueError("S3Storage.fetch_to_local requires a dst
path")
backend\storage\local.py:3:Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an
identity passthrough (the
backend\storage\local.py:30:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gcs.py:40:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
```

[Showing results with pagination = limit: 25]

53. user (2026-07-02T02:52:04.411Z)

```
60          )
61          return parse_uri(uri)
62
63
64      def input_is_local(raw: str, storage_config: Any = None) -> bool:
65          """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
66          plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
67          that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
68          ``resolve_input_ref``)."""
69          return resolve_input_ref(raw, storage_config).backend == "local"
70
71
72      def acquire_local_input(
73          raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
74      ) -> str:
75          """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
76
77          Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
78          byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
79          non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
80          path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
81          regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
82          worker's ``cmd_infer`` already ships."""
83          sc = _storage_dict(storage_config)
84          ref = resolve_input_ref(raw, sc)
85          if ref.backend == "local":
86              return (
87                  ref.key
88              )  # identity ? no temp, no copy, byte-identical to passing the path directly
89          scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
90          return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
91
92
93      def _backend_class(name: str):
94          """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
95          if name == "local":
96              from backend.storage.local import LocalStorage
97
98              return LocalStorage
99          if name == "s3":
```

54. user (2026-07-02T02:52:16.687Z)

No matches found

55. user (2026-07-02T02:52:18.046Z)

```
backend\main.py:646:        local = storage.acquire_local_input(rec["filepath"], getattr(cfg,
"storage", None))
backend\main.py:791:            local_working = storage.acquire_local_input(
backend\main.py:1168:    local_video = storage.acquire_local_input(
backend\main.py:1683:        local_src = storage.acquire_local_input(
```

```
backend\main.py:2056:        local_video = storage.acquire_local_input(args.video_path,
storage_cfg)
backend\config\models.py:378:    # scratch dir before processing (see
``backend.storage.acquire_local_input``).
backend\storage\__init__.py:33:    "acquire_local_input",
backend\storage\__init__.py:72:def acquire_local_input(
```

56. assistant (2026-07-02T02:52:26.716Z)

cleanup_caches doesn't cover rally-input-* scratch dirs. Let me confirm no caller deletes them.

57. user (2026-07-02T02:52:28.624Z)

```
1160
1161        notify = progress or (lambda stage: None)
1162
1163        notify("hashing")
1164        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
1165        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
1166        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
1167        # consumers (hash / validators / segmenter) use the resolved local path.
1168        local_video = storage.acquire_local_input(
1169            req.video_path, getattr(config, "storage", None)
1170        )
1171        video_id = compute_video_id(local_video)
1172        was_reingest = db.get_video(video_id) is not None
1173
1174        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
1175        notify("validating")
1176        logger.info(f"Running validations for {req.video_path}...")
1177        val_results, val_context = registry.run_all_with_context(local_video, config)
1178        failures = [r for r in val_results if not r.passed]
1179
1180        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
1181        default_seg = getattr(
1182            getattr(config, "indexing", None), "default_segmenter", "motion"
1183        )
1184        seg_name = req.segmenter or default_seg
1185        segmenter_actual = None
1186        segmentation_ran = False
1187        # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
1188        # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
1189        # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
```

58. user (2026-07-02T02:52:29.407Z)

```
1675        )
1676        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1677
1678        notify("stitching")
1679        try:
1680            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1681            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1682            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
```

```
1683            local_src = storage.acquire_local_input(
1684                video["filepath"], getattr(cfg, "storage", None)
1685            )
1686            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1687        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1688            logger.error(
1689                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1690            )
1691            return {
1692                "status": "failed",
1693                "message": f"Highlight stitching failed: {e}",
1694                "video_id": video_id,
1695            }
1696
1697        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1698        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1699        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1700        notify("finished")
1701        return {
1702            "status": "success",
1703            "filepath": out_path,
1704            "download_url": download_url,
```

59. user (2026-07-02T02:52:41.864Z)

```
4:        [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
173:    scratch: str,
184:        local = os.path.join(scratch, "_done.json")
209:    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
210:    os.makedirs(scratch, exist_ok=True)
211:    config.output.output_dir = scratch
213:    # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download to
scratch.
215:        args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
227:    db = Database(os.path.join(scratch, config.output.db_name))
241:                args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
309:    out_local = os.path.join(scratch, "outputs", video_id)
335:            args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
473:    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
474:    labels_dir = os.path.join(scratch, "labels")
475:    traj_dir = os.path.join(scratch, "trajectories")
476:    videos_dir = os.path.join(scratch, "videos")
576:    manifest_path = os.path.join(scratch, "manifest.json")
584:    out_dir = os.path.join(scratch, "artifacts")
599:                args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
654:        "--scratch", default=None, help="local scratch dir (default: a temp dir)"
706:    pt.add_argument("--scratch", default=None)
```

60. user (2026-07-02T02:52:55.545Z)

```
40              ffmpeg_bin(),
41              "-y",
42              "-i",
43              video_path,
44              "-vn",
45              "-ac",
46              "1",
47              "-ar",
48              str(sr),
49              "-c:a",
50              "pcm_s16le",
51              out_wav,
52          ]
```

```
53          subprocess.run(
54              cmd, check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
55          )
56          return out_wav
57
58
59      def load_wav(path: str) -> Tuple[np.ndarray, int]:
60          """Read a PCM WAV ? (float32 samples in [-1,1], sample_rate). Mono-downmixes."""
61          with wave.open(path, "rb") as w:
62              sr, n, sw, ch = (
63                  w.getframerate(),
64                  w.getnframes(),
65                  w.getsampwidth(),
66                  w.getnchannels(),
67              )
68              raw = w.readframes(n)
69          if sw == 2:
```

61. user (2026-07-02T02:52:56.326Z)

```
300                 "-preset",
301                 "superfast",
302                 "-crf",
303                 "22",
304                 "-c:a",
305                 "aac",
306                 "-b:a",
307                 "128k",
308             ]
309             vf_args = ["-vf", watermark_filter] if watermark_filter else []
310
311             # Try with the watermark first; if that encode fails (e.g. a bad font/logo
312             # path), retry once WITHOUT it so a branding misconfig can never nuke the
313             # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
314             attempts = [vf_args, []] if vf_args else [[]]
315             produced = False
316             last_exc: Optional[subprocess.CalledProcessError] = None
317             cmd = base_cmd + [clip_path]
318             for attempt_vf in attempts:
319                 cmd = base_cmd + attempt_vf + [clip_path]
320                 try:
321                     subprocess.run(cmd, capture_output=True, check=True)
322                 except subprocess.CalledProcessError as e:
323                     last_exc = e
324                     if attempt_vf and vf_args:
325                         _err = (
326                             e.stderr.decode(errors="replace").strip()[-300:]
327                             if getattr(e, "stderr", None)
328                             else ""
329                         )
330                         logger.warning(
331                             f"{segment_label}: watermark encode failed; retrying without it.
"
332                             f"ffmpeg: {_err or '(no stderr captured)'}"
333                         )
334                         continue
335                     break
336                 if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
337                     produced = True
338                     if not attempt_vf and vf_args:
339                         logger.warning(
```

62. user (2026-07-02T02:53:05.694Z)

```
89:| **O2** | MEDIUM | `backend/main.py` | Single `ThreadPoolExecutor(max_workers=1)` ? one
```

```
stuck job blocks all. | 2 |
90:| **O3** | MEDIUM | `backend/infrastructure/database.py` | SQLite WAL + busy_timeout but
concurrent writers (CLI + server) risk `SQLITE_BUSY`. | 3 |
91:| **O4** | LOW | `backend/pipeline/detectors/tracknet_runner.py` | `weights_path=''` ? silent
zero results. No startup weights-exist validation. | 3 |
202:| **P14** | **O2: Single-executor risk** ? documented with mitigations in job_worker.py | P7
| No | ? | [#374](https://github.com/Khelsutra/badminton-highlight-indexer/pull/374) |
205:| **P16+** | Phase 3 items (B4/B7/B8/D5/D6/D8-D10/D14/O3/O4) | P15 | No | ? | ? |
253:| 6+ | **P16+** | B4, B7, B8, D5, D6, D8?D10, D14, O3, O4 ? Phase 3 items |
255:> **Removed from this list ? already shipped (see the §6 tracker); they were stale entries
left from before Round 3/4 landed:** D4/P13 deep config merge (#373 ? `loader.py::_deep_merge`),
O2/P14 single-executor risk documented (#374 ? `job_worker.py`), Q7/P15 `motion.py` type-cleaned
(#373 ? no longer in the `mypy.ini` quarantine list).
278:[Omitted long matching line]
```

63. assistant (2026-07-02T02:53:58.406Z)

Let me spot-check the gemini chunk file cleanup and compiler concat line for citations.

64. user (2026-07-02T02:53:59.771Z)

```
210             t = 0.0
211             while t < video_duration:
212                 chunk_starts.append(t)
213                 t += step
214             num_chunks = len(chunk_starts)
215             logger.info(
216                 f"Video is {video_duration:.1f}s ? splitting into {num_chunks} overlapping
chunks "
217                 f"({chunk_duration}s each, {chunk_overlap}s overlap)."
218             )
219
220             os.makedirs(chunks_dir, exist_ok=True)
221
222             def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list, dict]:
223                 chunk_end = min(chunk_start + chunk_duration, video_duration)
224                 actual_chunk_dur = chunk_end - chunk_start
225                 chunk_label = f"Chunk {ci + 1}/{num_chunks}"
226                 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
227
228                 # Extract chunk with ffmpeg
229                 logger.info(
230                     f"Extracting {chunk_label}: {chunk_start:.1f}s - {chunk_end:.1f}s
({actual_chunk_dur:.1f}s)"
231                 )
232                 success = extract_video_chunk(
233                     video_to_upload,
234                     chunk_start,
235                     actual_chunk_dur,
236                     chunk_path,
237                     reencode=ai_scale_width > 0,
238                     scale_width=ai_scale_width if ai_scale_width > 0 else None,
239                 )
240                 if not success:
241                     logger.error(f"Failed to extract {chunk_label}. Skipping.")
242                     return chunk_label, [], {}
243
244                 # Measure chunk size
245                 try:
246                     chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
247                 except Exception:
248                     chunk_size_mb = 0.0
249
250                 # Get prompt from sport profile
251                 prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
```

```
252
253                 # Process this chunk through the provider
254                 chunk_segments, metrics = provider.analyze_video(
255                     chunk_path, prompt, chunk_duration=actual_chunk_dur, label=chunk_label
256                 )
257                 metrics["file_size_mb"] = chunk_size_mb
258                 metrics["segments_found"] = len(chunk_segments)
259
260                 # B2: surface provider failures so callers can distinguish "API error"
261                 # from "genuinely no rallies in this chunk" (both return [] today).
262                 if metrics.get("error"):
263                     logger.warning(
264                         "[%s] provider error ? chunk %d skipped: %s",
265                         chunk_label,
266                         ci,
267                         metrics["error"],
268                     )
269
270                 # Offset timestamps back to full-video coordinates
271                 for segment in chunk_segments:
272                     if "start_time" in segment:
273                         try:
274                             segment["start_time"] = (
275                                 float(segment["start_time"]) + chunk_start
276                             )
277                         except (ValueError, TypeError):
278                             pass
279                     if "end_time" in segment:
```

65. user (2026-07-02T02:54:00.965Z)

```
480                     ffmpeg_bin(),
481                     "-y",
482                     "-f",
483                     "concat",
484                     "-safe",
485                     "0",
486                     "-i",
487                     list_file_path,
488                     "-c",
489                     "copy",
490                     output_path,
491                 ]
492
493                 try:
494                     subprocess.run(concat_cmd, capture_output=True, check=True)
495
496                     # Final verification
497                     if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
498                         raise RuntimeError(
499                             f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
500                             f"but output file is missing or empty: {output_path}"
501                         )
502
503                     output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
504                     logger.info(
505                         f"Dynamic highlight reel saved: {output_path} ({output_size_mb:.1f}
MB)"
506                     )
507                     return output_path
508                 except subprocess.CalledProcessError as e:
509                     stderr_msg = e.stderr.decode(errors="replace").strip()
```

66. user (2026-07-02T02:54:08.661Z)

```
129:        Returns (video_to_upload, cleanup_temp_files) where cleanup_temp_files removes any
147:                    os.remove(temp_trimmed_path)
183:        def cleanup_temp_files():
186:                    os.remove(temp_trimmed_path)
190:        return video_to_upload, cleanup_temp_files
202:        chunks_dir: str,
203:        cleanup_temp_files: Callable[[], None],
220:        os.makedirs(chunks_dir, exist_ok=True)
226:            chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
289:                os.remove(chunk_path)
375:        cleanup_temp_files()
377:            os.rmdir(chunks_dir)
394:        cleanup_temp_files: Callable[[], None],
428:        cleanup_temp_files()
630:        # Optionally trim the video first (debug feature); cleanup removes any temp file.
631:        video_to_upload, cleanup_temp_files = self._maybe_trim_video(
655:        # video keeps its own dir, so cleanup here never removes another run's chunks.
656:        chunks_dir = _chunk_workdir(video_id, output_base(self.config))
668:                chunks_dir,
669:                cleanup_temp_files,
678:                cleanup_temp_files,
```

67. assistant (2026-07-02T02:54:33.105Z)

I have all the evidence needed. Compiling the final audit report.

68. user (2026-07-02T02:55:31.814Z)

Structured output provided successfully